

Actividad: Condición de Carrera y Sección Crítica

Asignatura: Concurrencia y Paralelismo

Estudiante: María F. Suárez Arias

1. Código Fuente del Programa (codigo_fuente.c)

A. Código SIN Sincronización (Demuestra Condición de Carrera)

```
#include <stdio.h>
#include <pthread.h>

int global_counter = 20;

void *thread_routine(void *arg) {
    for (size_t i = 0; i < 1000; i++) {
        global_counter++; // Sección crítica no protegida
    }
    return NULL;
}

void *thread_routine_two(void *arg) {
    for (size_t i = 0; i < 1000; i++) {
        global_counter--; // Sección crítica no protegida
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_routine, NULL);
    pthread_create(&t2, NULL, thread_routine_two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Valor final de global_counter: %d\n", global_counter);
    return 0;
}
```

```
}
```

B. Código CON Sincronización (Utilizando Mutex)

```
#include <stdio.h>
#include <pthread.h>

int global_counter = 20;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *thread_routine(void *arg) {
    for (size_t i = 0; i < 1000; i++) {
        pthread_mutex_lock(&mutex);    // Inicio de Sección Crítica
        global_counter++;
        pthread_mutex_unlock(&mutex); // Fin de Sección Crítica
    }
    return NULL;
}

void *thread_routine_two(void *arg) {
    for (size_t i = 0; i < 1000; i++) {
        pthread_mutex_lock(&mutex);    // Inicio de Sección Crítica
        global_counter--;
        pthread_mutex_unlock(&mutex); // Fin de Sección Crítica
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_routine, NULL);
    pthread_create(&t2, NULL, thread_routine_two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Valor final de global_counter: %d\n", global_counter);
    return 0;
}
```

2. Explicación de Conceptos Teóricos

¿Qué es una condición de carrera (Race Condition)?

Una condición de carrera es cuando dos o más procesos intentan usar o modificar el mismo dato al mismo tiempo, y el resultado depende de quién llega primero.

¿Qué es una sección crítica y qué parte del código corresponde a ella?

La sección crítica es el fragmento de código donde se accede o modifica un dato compartido que no debe ser manipulado al mismo tiempo por más de un hilo para evitar inconsistencias. En el programa, las secciones críticas corresponden exactamente a las líneas donde se altera el contador:

- `global_counter++;` (en `thread_routine`)
- `global_counter--;` (en `thread_routine_two`)

3. Fragmento y Análisis del Código Ensamblador (`race_test.s`)

Fragmento extraído de las rutinas de hilo donde se identifican las instrucciones máquina:

; Fragmento de incremento (Hilo 1)

.L3:

```
movl  global_counter(%rip), %eax ; Carga global_counter a registro %eax
addl  $1, %eax                  ; Suma 1 en el registro
movl  %eax, global_counter(%rip) ; Guarda el resultado en memoria
```

; Fragmento de decremento (Hilo 2)

.L7:

```
movl  global_counter(%rip), %eax ; Carga global_counter a registro %eax
subl  $1, %eax                  ; Resta 1 en el registro
movl  %eax, global_counter(%rip) ; Guarda el resultado en memoria
```

Instrucción	Significado	Función en global_counter
movl	Move Long (32 bits)	Copia el valor desde la memoria RAM al registro (%eax) o escribe el resultado de vuelta a la memoria.
addl	Add Long (Suma)	Suma 1 al valor en el registro (global_counter++).
subl	Subtract Long (Resta)	Resta 1 al valor en el registro (global_counter--).

4. Resultados de Experimentación y Evidencias

Caso A: Sin Sincronización

Al ejecutar el programa sin el candado mutex, el resultado fue inconsistente tras múltiples ejecuciones consecutivas en PowerShell, registrándose valores como **20** y **11**. Esto demuestra la presencia de condiciones de carrera.

```

Windows PowerShell
PS C:\Users\maria\Downloads> gcc -pthread race_test.c -o race_test
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 11
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> gcc -S race_test.c
PS C:\Users\maria\Downloads>

```

Caso B: Con Sincronización (Mutex)

Al proteger las secciones críticas mediante pthread_mutex_lock y pthread_mutex_unlock, el

resultado obtenido en todas las ejecuciones de prueba fue consistentemente el valor esperado: 20.

```
PS C:\Users\maria\Downloads> gcc -pthread race_test.c -o race_test
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> ./race_test
Valor final de global_counter: 20
PS C:\Users\maria\Downloads> ./race_test
```

5. Conclusión Breve

A través de esta práctica pude comprobar que las operaciones de aumentar o disminuir una variable en C no se realizan de una sola vez, sino que primero se lee el valor, después se modifica y finalmente se guarda. Cuando varios hilos trabajan al mismo tiempo sin ningún tipo de control, pueden ocurrir errores porque un hilo puede modificar el valor mientras otro también lo está haciendo. Esto puede provocar que se pierdan algunos cambios y que el resultado final no sea el esperado.

Para solucionar este problema utilicé `pthread_mutex`, que permite que solo un hilo pueda modificar la variable a la vez. De esta manera, pude evitar que los hilos interfirieran entre sí y logré que los datos se mantuvieran correctos.